```swift
/*:
## App Exercise – Printable Workouts

 >These exercises reinforce Swift concepts in the context of a fitness
  tracking app.

 The `Workout` objects you have created so far in app exercises don't show a
  whole lot of useful information when printed to the console. They also
  aren't very easy to compare or sort. Throughout these exercises, you'll make
  the `Workout` class below adopt certain protocols that will solve these
  issues.
 */
class Workout: CustomStringConvertible, Equatable, Comparable, Codable  {
    var distance: Double
    var time: Double
    var identifier: Int

    init(distance: Double, time: Double, identifier: Int) {
        self.distance = distance
        self.time = time
        self.identifier = identifier
    }
        var description: String {
            return ("You went: \(distance) feet in \(time) minutes!")
        }

    static func ==(lhs: Workout, rhs: Workout) -> Bool {
        return lhs.identifier == rhs.identifier
    }

    static func <(lhs: Workout, rhs: Workout) -> Bool{
        return lhs.distance > rhs.distance && lhs.time < rhs.time
    }

}

let w1 = Workout(distance: 50.2, time: 10.5, identifier: 1)
let w2 = Workout(distance: 100.2, time: 30.2, identifier: 2)
let w3 = Workout(distance: 80.0, time: 20.5, identifier: 3)
let w4 = Workout(distance: 30.2, time: 15.2, identifier: 4)
let w5 = Workout(distance: 150.2, time: 50.3, identifier: 5)


//:  Make the `Workout` class above conform to the `CustomStringConvertible`
 protocol so that printing an instance of `Workout` will provide useful
 information in the console. Create an instance of `Workout`, give it an
 identifier of 1, and print it to the console.
print(w1)
```

```
//:  Make the `Workout` class above conform to the `Equatable` protocol. Two
 `Workout` objects should be considered equal if they have the same
 identifier. Create another instance of `Workout`, giving it an identifier of
 2, and print a boolean expression that evaluates to whether or not it is
 equal to the first `Workout` instance you created.
print (w1==w2)

/*:
 Make the `Workout` class above conform to the `Comparable` protocol so that
  you can easily sort multiple instances of `Workout`. `Workout` objects
  should be sorted based on their identifier.

 Create three more `Workout` objects, giving them identifiers of 3, 4, and 5,
  respectively. Then create an array called `workouts` of type `[Workout]` and
  assign it an array literal with all five `Workout` objects you have created.
  Place these objects in the array out of order. Then create another array
  called `sortedWorkouts` of type `[Workout]` that is the `workouts` array
  sorted by identifier.
 */
let workouts = [w1, w2, w3, w4, w5]
let sortedWorkouts = workouts.sorted(by: <)

//:  Make `Workout` adopt the `Codable` protocol so you can easily encode
 `Workout` objects as data that can be stored between app launches. Use a
 `JSONEncoder` to encode one of your `Workout` instances. Then use the encoded
 data to initialize a `String`, and print it to the console.
import Foundation
let jsonEncoder = JSONEncoder()
if let jsonData = try? jsonEncoder.encode(workouts[2]), let jsonString =
 String(data: jsonData, encoding: .utf8){
    print(jsonString)
}

/*:
[Previous](@previous)  |  page 2 of 5  |  [Next: Exercise - Create a
 Protocol](@next)
 */
```